svngod

面向大规模、强一致场景的 SVN 兼容版本服务器

一次提交 3 万个文件，1GiB 二进制流式处理，WAL + 法定人数保证多副本崩溃原子性 —— 原生 svn 客户端零改造接入。

2026 年 11 月 · 对比 Apache mod_dav_svn

1.2s

30,000 个文件一次原子提交

约 100 MiB/s

1GiB 上传（CDC 分块）

1 个节点

3 节点法定人数下仍可提交

CRC32C WAL

每次提交可审计、可前滚

原生 SVN 的三道坎

Apache + mod_dav_svn 成熟可靠，但它的提交模型诞生于“一次提交几十个文件”的年代。当仓库变成游戏资产、固件、模型或 monorepo，瓶颈会同时出现在三处。

- **每文件一次 HTTP 往返**：提交 n 个文件要走 n 次 PUT 加活动/事务协商；1 万文件意味着上万次往返，客户端与服务端都容易在 600 秒网关超时前断流。
- **大文件依赖“小心调优”**：单文件数 GiB 时，缓冲区、超时、临时空间与内存峰值相互牵制，稍有不慎就是 OOM 或半途失败。
- **单机语义**：FSFS 天然单机；要做多副本、跨站点容灾，通常得叠加共享存储、DRBD 或自研同步，一致性边界模糊。

> **重点｜ svngod 保留 SVN 的协议、客户端与心智模型，重写的是提交通道、存储引擎与一致性层。**

七项可量化的优势

- **一次请求，一个原子修订**：svngocli 的 bundle 通道把整棵目录树打包成一个请求；30,000 个文件一次提交约 1.2 秒，服务端只产生一个修订，失败则整笔不落库。重复发布先协商 stat，只传变化文件；确实变化的大文件按块清单只补传新块（8MiB 文件改 1MiB，请求体约 2MiB）。
- **大文件分块去重，内存有界**：大文件按内容定义切块（平均 1MiB，可按仓库配置），1GiB 上传约 10 秒（约 100 MiB/s），读取 648 MiB/s，峰值内存约 20 MiB；随机改写 10MiB 只新增约 12MiB。
- **HTTPv2 默认开启**：与现代 svn 客户端自动协商，一次提交从“活动/检出的多轮往返”缩减为 create-txn + PUT + MERGE；`SVNGOD_HTTPV2=0` 可回退 v1。
- **WAL + 法定人数（2026-11）**：每次提交先把带 CRC32C 的记录写到 `journal/<N>`，再推进 HEAD；崩溃后可判定“是否已发布”，并在启动/下次提交时把已达成法定人数的记录**前滚**。严格模式 `quorum: 2`（3 节点）下，写不够多数派就返回 503，绝不单副本确认。
- **崩溃原子性**：跨进程提交租约 + WAL 对账，token 幂等重试；上传中杀掉服务进程或提交中杀掉客户端，都不会留下半个修订；陈旧节点恢复后不会复活旧 HEAD 或重复修订号。
- **多副本与主备镜像**：本地盘或 S3 后端；默认主备可用性模式（**备节点故障不停写；主节点故障时提交 503、读继续**），可选法定人数严格模式（RPO=0）；主备双写、故障切换与秒级自愈修复。
- **标准互操作**：dump 为 SVN dump v2 格式，原生 `svnadmin load` 可直接加载并 verify；authz、hooks、TLS 与原生 svn 客户端全部沿用。

实测数据：批量提交

场景	原生 svn 客户端 → svngod	svngocli bundle	结果
1,000 个文件（约 4MB）	1.38s	0.06s（约 22×）	各 1 个修订，内容校验一致
30,000 个文件（300 目录）	35s	1.2s	30,300 项齐全，verify 干净

原生 Apache 的提交同样是逐文件 PUT；文件数上万时往返次数线性增长，现场常见 600 秒网关超时。svngod 的原生客户端路径 35 秒完成，bundle 路径把往返压到 1 次。

组提交（CommitBatch）：批量写入把同一租约周期的多个修订合并为一次对象写入与一次 HEAD CAS——单进程 K=64 约 33 次/秒，8 个仓库并发 45–49 次/秒；16 仓 × 625,000 修订的千万级长跑约 60 小时。在线 DAV 提交默认开启组提交（16 批 / 2ms 窗口），但原生客户端受 MKACTIVITY/PUT/MERGE 会话往返限制，8 并发约 4.5 次/秒。

实测数据：大文件（1GiB / 2GiB 随机二进制）

指标	svngod	Apache mod_dav_svn
1GiB 提交（bundle，分块）	约 10s / 约 100 MiB/s	—
1GiB 提交（原生 svn，分块）	16.6s / 62 MiB/s	7.6s / 134 MiB/s
服务端内存峰值	约 20 MiB	未测
1GiB 读取（分块流式）	648 MiB/s	—
1GiB 文件占盘	1.1GB	3.1GB

单次端到端测量，含客户端读盘与哈希；所有文件以 sha256 回读校验，仓库 verify 无完整性问题。

实测数据：1GiB 文件随机改写 10MiB

指标	svngod（CDC 分块）	Apache mod_dav_svn
提交耗时	17.1s	10.9s
仓库增长	+11.8MiB	+10.2MiB
服务端内存峰值	约 69 MiB	未测

单文件随机改写上，原生 FSFS 的精确 delta 仍然更快、更省；CDC 的价值在跨文件/跨修订去重、按需范围读取与无链式重建。两者都远优于默认按全量副本存储二进制的方案。

实测数据：WAL、法定人数与多进程（RustFS 1.0.4）

场景	结果
RustFS 条件写原子性（8 并发 CAS）	仅 1 个成功，其余 PreconditionFailed
3 节点 `quorum:2`，1 个节点宕机	提交成功（首次切换约 11s，标记降级后 1s）
3 节点 `quorum:2`，2 个节点宕机	1s 内返回 503，HEAD 不变
节点恢复后重试	1s 内提交成功，WAL 对账，无重复修订
两个 svngod 进程 6 并发提交	6/6 成功，HEAD 精确 +6，6 个唯一修订
陈旧节点（落后 2 个修订）重新加入	下次提交先对账再提交，三节点最终一致
2 节点主备，备节点宕机	提交 1s，head +1
2 节点主备，主节点宕机	3–5s 内提交返回 503（HEAD 不变）；读切备继续，主节点恢复后重试成功
主节点在上传 100MiB 增量时被杀	上传被丢弃，不产生半个修订；恢复后重试成功（3 节点下可切备继续，sha256 一致）
两个存储节点同时宕机	干净失败（HEAD 不变），恢复后重试成功
三节点独立 verify	同一修订，均无完整性问题

两节点主备：备节点故障时提交继续（写入只落主节点，恢复后自动补齐）；主节点（列表第一项）故障时提交返回 503、HEAD 不变，读仍可继续。要“主节点故障仍可提交”，用 3 节点 + `quorum:2`（RPO=0，容忍 1 节点故障）。

实测数据：故障注入与一致性

场景	结果
30,000 文件上传中 kill -9 服务副本	无修订发布；verify 干净；重启后重传成功
提交 200MB 文件中 kill -9 客户端	无修订发布；cleanup 后提交成功
跨副本事务（A 建、A/B 写、B 提交）	成功，双文件一致
S3 主库 5% 错误注入	上传成功，verify 干净
S3 主库宕机、恢复	上传成功；10 秒完成镜像收敛
S3 备库宕机、恢复	上传成功；17 秒完成镜像收敛
双副本并发 20 次提交	全部成功，无 CAS 冲突
3 轮混合压测	成功提交数与修订数精确对账，全部 verify 干净

实测数据：标准格式互操作

项目	结果
svngod dump → 原生 svnadmin load	146MB / 82 修订；62,741 项完全一致；原生 verify 通过
S3 镜像 dump → 原生 svnadmin load	19,653 项完全一致
HTTPv2 协商	svn 1.14 自动启用；可回退 v1
原生客户端功能	checkout / add / commit / update / log / diff / lock 无需改造

定位对比

维度	Apache mod_dav_svn	svngod
提交模型	每文件一次 HTTP 往返 + 活动/事务协商	目录树一次请求（bundle）或 HTTPv2 事务
万级文件提交	往返线性增长，易触发网关超时	30,000 文件 1.2 秒
大文件	可行，依赖部署调优	1–2GiB 流式，内存 15–26 MiB
二进制增量	支持，单文件小编辑更快	支持，链深有界、自动压平
多副本一致性	依赖共享存储或外部同步	WAL + 法定人数 + 跨进程租约；可选 RPO=0
多进程/多活	需外部协调	两个 svngod 进程可共享镜像，并发提交唯一修订号
存储后端	本地 FSFS	本地盘 / S3（RustFS/MinIO/AWS）；主备自愈
互操作	SVN 生态标准	原生客户端 + 标准 dump
运维形态	Apache + 模块 + 调优	单 Go 二进制 + Prometheus 指标

零改造迁移

保留现有 svn 客户端、authz 文件、钩子与工作流；服务端替换为 svngod 即可。

```
svngod --addr :8080 --repo-root /var/lib/svngod \
  --passwd /etc/svngod/passwd --authz /etc/svngod/authz

# 现有的 svn 客户端照常工作
svn co http://host:8080/svn/repo/trunk

# 需要批量发布时，用增强客户端一次提交整棵树
svngocli upload ./assets https://host/svn/repo/assets -m "release"
```

存量仓库可通过标准 dump/load 迁移；svngod 的 dump 也能被原生 `svnadmin` 加载，迁移与回退都不锁定。

> **可选：svngod 直接对接 S3（RustFS / MinIO / AWS S3），用主备镜像替代共享存储。**

适用场景

- **游戏与数字资产**：大量中小文件、频繁整包发布、需要一次原子修订。
- **固件与二进制制品**：单文件数 GiB，要求流式与内存可控。
- **monorepo / CI 高频提交**：万级文件提交不超时，失败可幂等重试。
- **多站点与容灾**：S3 主备、跨副本提交、崩溃后无半修订；RPO=0 可用 3 节点法定人数。
- **既想保留 SVN 生态，又需要现代化存储与一致性。**

结语

svngod 不是另起炉灶的版本控制系统，而是把 SVN 的提交通道与存储引擎换成适配 2026 年的实现：协议不变、客户端不变、运维更轻，批量、大文件、一致性与容灾都有可复现的数据。

仓库：`https://github.com/raindylong/svngod`

> **重点｜ 30,000 个文件 1.2 秒，1GiB 约 10 秒，内存 17MiB；WAL + 法定人数让“快”和“不丢”同时成立。**

测试环境与口径

> **Linux x86_64，本地 overlay 盘；svn 1.14.1；Apache 2.4.52 + mod_dav_svn；RustFS 1.0.4（单机双节点/三节点实例）；svngod master（2026-11，含 P0/P1 与 WAL/法定人数）。大文件为 /dev/urandom 随机二进制。所有数据以 sha256 回读、仓库 verify 双重校验。吞吐受硬件、网络与并发影响，生产容量请以自有环境压测为准。对比对象为 Apache mod_dav_svn；svngod 同时兼容标准 SVN dump 与原生客户端。**